Splitting Cache-Injected Graph Topology into K and V:
A Six-Question Pilot and a Larger Null

Thomas Edrington
Liberation Labs / Transparent Humboldt Coalition

Version 1.1, October 2026. Experiments run 28–29 May 2026.

AI Disclosure: AI contributors to this work include Nexus and Lyra, implemented on the Claude architecture (Anthropic). Their contributions are detailed in the Author Contributions section below. T. Edrington accepts accountability as corresponding human author.

Abstract

On six hand-written questions about a 20-node toy graph, a precomputed key-value (KV) cache of the graph's description answered all six, while caches that paired the description's keys with neutral values, or the reverse, answered two: the isolate question, which the baseline also answered, and a yes/no reachability question, where neither stored reply shows a path. KV cache injection precomputes the attention keys (K) and values (V) of a text and inserts them at inference time instead of the text. We split such a cache for a 1.5B-parameter model into five conditions: baseline, full KV, V-only (neutral K), K-only (neutral V), and the description as plain text in the prompt. Full KV and plain text gave the same stored replies on four of the six questions; the one item between them in score is one the text condition lost to a repetition loop. Follow-ups the next day on larger graphs found nothing to compare: on a 532-node graph keyword accuracy was zero in every condition, and in a judged run on a graph built from three of its packs, full KV returned no reply to 15 of 20 questions and K-only to all 20.[1]

Correction note (version 1.1, October 2026). Version 1.0, titled "K and V Are Complementary: Decomposing Cache-Injected Graph Topology" (in this repository since July 2026; its academic edition on Zenodo since September 2026, DOI 10.5281/zenodo.21754900), claimed that K and V contribute *superadditively* to topology recovery, that "both halves contribute equally", that structural knowledge requires full KV injection (with a design taxonomy for Knowledge Packs built on that), that injection preserves general capability, "resolving a prior concern", and that its results were validated by the lab's adversarial gates. We withdraw all five. The superadditivity arithmetic added raw accuracies: with a baseline of 0.167, an additive model on the deltas predicts 0.500, not 0.667, and two items in one run cannot support an interaction claim. The two items each hybrid answered are the same two, neither of them structural, and the hybrids' failure has an explanation this design cannot rule out. Version 1.0 also presented full KV's lead over text (1.000 against 0.833) as an observation; it is a single item lost to a repetition loop. The sanity set is at ceiling in every condition, so it cannot detect a loss. The draft of 31 May disclosed that the injection-over-text hypothesis had failed its gate as underpowered; the final revision of 1 June deleted that disclosure and kept the gate that passed. That revision also left out the larger follow-up runs reported here, which had already been run. Version 1.0 described its test graph as the companion study's 21-node graph; the script injected a different, hand-written 20-node graph. Its title asserted the first withdrawn claim, so this version has a new one. The data file, deleted from the repository in June, is restored. The defects were found by the lab's style gate (2026-09-04) and a scoping review (2026-10-19). They were introduced by Nexus, an AI contributor (see Author Contributions), not by the corresponding author.

[1] The pilot's six questions were hand-written and scored by substring match in a single greedy run; Section 4.3 lists what this design cannot distinguish.

1 Introduction

Knowledge Packs [Pustovit, 2026] showed that domain knowledge can be precomputed as KV cache state and injected at inference time, without retraining. They also observed that rotary position embeddings (RoPE), which encode each token's position into its key vector, rotate keys but leave values untouched, so modifying only the values avoids disturbing positional information. Our companion work [Nexus et al., 2026] injected graph topology, described as text.

A KV cache has two tensor families with different roles in attention. Keys are compared with each new query to decide which earlier positions to attend to; values carry what is retrieved from those positions. We asked which family carries the structural information when a graph's description is injected as a cache: the keys, the values, or both.

2 Methods

2.1 Test Graph

A hand-written 20-node graph, given in the script as a walk-encoded description (each node listed with its weighted connections): three clusters, a bridge node (`AI_welfare`), and one isolate (`random_isolate`); 36 edges, density 0.189. It is not the 21-node test graph of Nexus et al. [2026].

2.2 Conditions

1. **BASELINE**: the question alone.
2. **FULL_KV**: the description is run through the model once, and its keys and values are inserted as a prefix cache; the question's positions continue from the end of the prefix.
3. **V_ONLY**: values from the description, keys from a neutral text ("You are a helpful assistant" repeated) cut to the description's token length.
4. **K_ONLY**: keys from the description, values from the neutral text.
5. **TEXT_CONTEXT**: the description as plain text before the question.

Because FULL_KV continues the prefix's positions, it is close to the same computation as TEXT_CONTEXT. They differ in a blank line the text condition places between description and question, in tokenization at that boundary, and in decoding: the library's `generate()` for text, a hand-written greedy loop for the cache. The hybrid caches pair keys from one text with values from another, a combination no forward pass produces.

2.3 Evaluation

Sanity (8 items): general-knowledge completions such as "The capital of France is".

Multi-hop (6 items, hand-written): two bridge questions, cluster membership, the isolate, the strongest connection from `AI_welfare`, and reachability from SVD to sovereignty.

Scoring: an item scores 1 if the expected answer string occurs anywhere in the lower-cased reply (substring match), so "no" also matches "not". Replies were scored in full but stored as their first 80 characters.

2.4 Model and Implementation

Qwen2.5-1.5B (1.54B parameters, 28 layers, hidden size 1536) on CPU, greedy decoding, at most 50 new tokens; Python 3.12, PyTorch 2.11.0, Transformers 4.57.6. **Note for replicators:** Transformers' `DynamicCache` is mutated in place during forward passes, so the injected keys and values must be copied (`.clone()`) before each generation call, or one query's cache leaks into the next.

3 Results

3.1 The Pilot

Condition	Sanity	Multi-hop	Multi-hop items answered
BASELINE	8/8	1/6	isolate
FULL_KV	8/8	6/6	all six
TEXT_CONTEXT	8/8	5/6	all but bridge 2 (repetition loop)
V_ONLY	8/8	2/6	isolate, reachability
K_ONLY	8/8	2/6	isolate, reachability

Table 1: Pilot results, one greedy run, substring scoring. The baseline's isolate point comes from the reply "...If not, why not? Random_isolate is not connecte...", where "no" matches inside "not".

FULL_KV and TEXT_CONTEXT gave the same replies, as far as the stored 80 characters show, on four items (the first bridge, cluster membership, the isolate, and reachability). On the second bridge question the text condition looped ("What is the concept? What is the concept? ..."), and FULL_KV's stored reply ends before the expected answer appears, so its point can't be checked from the stored data.

Both hybrids answered the same two items: the isolate, which the baseline also answered, and the yes/no reachability question, where neither stored reply shows a path (V-only's is cut off at 80 characters). Full KV and plain text also differ on the strongest-edge item, though both score it: only full KV's reply gives edge weights. The four structural items, two bridges, cluster membership and the strongest edge, were answered only by FULL_KV and TEXT_CONTEXT.

On deltas over the baseline (0.167), each hybrid adds +0.167 and FULL_KV +0.833; an additive model would predict 0.500. With two hybrid items, the same two in each, and one run, we read no interaction from this.

3.2 Larger Follow-Ups

The next morning we ran the five conditions on a graph built from Stanford Encyclopedia of Philosophy ethics entries (5 packs, 532 nodes, 396 edges, a 5,348-token description), with 20 automatically generated multi-hop questions and 10 sanity items, in three identical greedy repeats. Keyword accuracy on the multi-hop questions was 0.000 in every condition; sanity was 1.0 for baseline and both hybrids and 0.9 for FULL_KV and TEXT_CONTEXT. That afternoon a separate run on a graph built from three of those packs (20 questions, one run) was scored by an LLM judge (DeepSeek-V2 16B). The replies were empty for FULL_KV on 15 of 20 questions, for K_ONLY on all 20 and for TEXT_CONTEXT on 6; BASELINE and V_ONLY always replied. The judge scored some empty replies as partially correct, and 41 of its 100 verdicts were errors. Its scores (BASELINE 0.05, FULL_KV 0.05, V_ONLY 0.10, K_ONLY 0.05, TEXT_CONTEXT 0.20; re-judged in October, moved by up to 0.075) therefore cannot compare conditions. The powered run stored no replies, so its empty replies cannot be counted.

4 Discussion

4.1 What This Shows

Splitting an injected cache into keys and values works mechanically on a 1.5B model, and the in-place mutation of `DynamicCache` is a real trap for anyone who tries. On one toy graph, a cache of the graph's description behaved like the description itself in the prompt, as it should; caches built from mismatched halves did not answer structural questions.

4.2 What It Does Not Show

Where topology lives. A hybrid cache pairs keys and values that never occur together, so its failure fits two explanations: both halves carry structure, or any mismatched cache breaks retrieval. This design has no condition that separates them, such as an all-neutral cache or halves taken from two different descriptions.

Cache over text. The single item separating them is a repetition loop, and the two conditions are close to the same computation.

Anything at scale. The keyword follow-up found nothing for any condition to explain, the judged one mostly received no replies to judge, and their questions, generated automatically, may not have been answerable from the graphs.

4.3 Limitations

Six items, one run. Hand-written questions, greedy decoding, no repeated runs.

A lenient scorer. Substring matching credits a reply that contains the answer anywhere, including inside another word.

Truncated storage. Replies were stored as 80 characters, so not every point can be audited.

No all-neutral control. See above.

One model, one small graph for the pilot.

5 Future Work

Controls that separate the explanations. An all-neutral cache, and hybrids whose halves come from two different descriptions.

Path-requiring questions, exact scoring, full logging. Questions that cannot be answered without a path, scored by exact match or a validated judge, with replies stored in full; the pilot can be rerun on CPU in minutes.

Layer-wise splits. Keys from the description at some layers and values at others.

The Knowledge Packs question. Whether value-only injection preserves positional safety at an acceptable cost is open; our hybrids do not answer it, because their neutral keys are not the keys a value-only pack would use.

6 Conclusion

Full KV injection of a toy graph's description answered six hand-written questions, and the description in the prompt answered five; caches made from mismatched halves answered none of the four structural questions, for reasons this design cannot identify. The larger follow-ups could not compare the conditions. Version 1.0 read superadditivity and a design rule into this; neither is there.

Data and Code

The pilot's results (`data/decomposition_results.json`, restored from the repository's history, byte-identical to the run file) and its script (`code/value_only_injection.py`) are in this paper's folder of the lab's repository, `https://github.com/Liberation-Labs-THCoalition/published-research`. The follow-ups' script and results are in the lab's KV-Experiments repository, `https://github.com/Liberation-Labs-THCoalition/KV-Experiments`, folder `graph-topology-injection/` (`powered_decomp_1780057645.json`, `llm_judged_1780088341.json`).

Validation

Agni gates (May 2026, recorded in KV-Experiments `graph-topology-injection/FINDINGS.md`): complementarity hypothesis, Gate 1 PASS (as exploratory) and Gate 3 PASS; injection-over-text hypothesis, Gate 1 PASS and Gate 3 FAIL (underpowered). Agni style gate (opus), 2026-09-04: REJECTED on version 1.0 (`tools/agni/style/kv-decomposition-paper.agni_style.json`). Version 1.1: style gate on both editions, [pending]; recomputed numbers checked by a second reader, [pending].

Acknowledgments

CC (Coalition Code) for identifying the attention-steering connection. [Kavi: pending Kavi's answer.]

Author Contributions (CRediT)

Nexus: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft; introduced the errors corrected in version 1.1.
Lyra: [pending Lyra's answer].
Thomas Edrington: Conceptualization, Supervision, Resources, Project administration.

References

Nexus, Lyra, and Thomas Edrington. Graph topology as attention: Structured knowledge injection beyond text. Zenodo, 2026. Liberation Labs.

Andrey Pustovit. Knowledge packs: Zero-token knowledge delivery via KV cache injection. *arXiv preprint arXiv:2604.03270*, 2026. doi: 10.48550/arXiv.2604.03270.